全国大学生嵌入式芯片与系统设计竞赛

FPGA 创新设计赛道 · 设计报告

FPGA 运动编码器在线状态监测系统

赛题 A 题 · 平台 EGO1（Xilinx Artix-7 XC7A35T-1CSG324C）

组　　号	51528
成　　员	胡毅（25019100600） 贺宇翔（25019100088） 杨海涛（25019100022）
班　　级	2501015
开发工具	Vivado 2026.1 纯 Verilog HDL（无软核、无 SDK、无外部 MCU）
报告日期	2026 年 11 月 3 日

目　　录

目录 4

1. 作品概述 4

2. 赛题要求对照 5

3. 系统总体架构 5

4. 硬件平台与引脚分配 6

4.1 平台 6

4.2 引脚分配（ego1.xdc） 6

5. 各模块设计与方案论证 7

5.1 enc_sync —— 两级同步 7

5.2 enc_debounce —— 计数式消抖 7

5.3 enc_quad —— 正交解码（4 倍频） 7

5.4 enc_pos —— 位置与 Z 相校准 8

5.5 enc_speed —— M 法测速 8

5.6 enc_filter —— 4 点滑动平均 9

5.7 enc_alarm —— 三类报警 9

5.8 UART 上传链路 9

5.9 OLED 显示链路 10

6. 关键时序设计 11

6.1 系统时间基准 11

6.2 数码管扫描时序 11

6.3 UART 帧与数据流的相位关系 11

7. 资源占用与时序收敛 12

7.1 资源占用（实现后） 12

7.2 时序收敛（100 MHz 主时钟，周期 10 ns） 12

7.3 产物 12
8. 性能指标与量程标定 12
8.1 高速上限：25000 转/分 12
8.2 低速性能（修复两级取整后） 13
8.3 发现并修复的 bug：两级向下取整 13
9. 仿真验证证据 14
9.1 分层单元测试 14
9.2 集成测试（tb_top_uart.v） 15
9.3 板级整链测试（tb_board_top.v） 15
9.4 板级仿真抓出的两个真问题 15
10. 上位机软件 16
10.1 功能 16
10.2 关键设计与验证 16
11. 创新点总结 16
12. 上板验证与真机实测 17
12.1 烧录与脱机运行 17
12.2 上电判据（三条，缺一不可） 17
12.3 编码器接线与供电 17
12.4 真机实测结果 18
12.5 真机联调抓出的三个 bug（均已修复） 19
12.6 八条赛题要求的真机验证状态 19
13. 已知限制与改进方向 19
附录 A：源文件清单 20

赛题：A 题　作品名：FPGA 运动编码器在线状态监测

平台：EGO1（Xilinx Artix-7 **XC7A35T-1CSG324C**）　工具：Vivado 2026.1

实现语言：纯 Verilog HDL（**无软核、无 SDK、无外部 MCU**）

报告日期：2026-11-03（提交版）

目录

1. [作品概述](#1-作品概述)
2. [赛题要求对照](#2-赛题要求对照)
3. [系统总体架构](#3-系统总体架构)
4. [硬件平台与引脚分配](#4-硬件平台与引脚分配)
5. [各模块设计与方案论证](#5-各模块设计与方案论证)
6. [关键时序设计](#6-关键时序设计)
7. [资源占用与时序收敛](#7-资源占用与时序收敛)
8. [性能指标与量程标定](#8-性能指标与量程标定)
9. [仿真验证证据](#9-仿真验证证据)
10. [上位机软件](#10-上位机软件)
11. [创新点总结](#11-创新点总结)
12. [上板验证方案](#12-上板验证方案)
13. [已知限制与改进方向](#13-已知限制与改进方向)

1. 作品概述

本系统面向旋转机械的**在线状态监测**需求，以单片 FPGA 完成从编码器信号采集到状态显示与上传的**全链路硬件化**处理。系统采集增量式旋转编码器的 A/B/Z 三相信号，经同步与消抖后做正交解码，实时解算出**位置（圈数 + 圈内角度）**与**瞬时转速**，经硬件滑动平均滤波后，同时完成三件事：

· **本地可视化**：8 位数码管显示位置与转速；OLED（SSD1306，128×64）滚动绘制转速波形
· **本地报警**：超速、丢脉冲、非法跳变三类异常由 LED 指示并锁存
· **远程上传**：115200 bps UART 以定长帧上传，PC 上位机实时解析、绘图、记录

所有算法（同步、消抖、解码、测速、滤波、报警、I2C 主机、UART 发送、数码管扫描、OLED 显存与波形绘制）**全部由 RTL 硬件逻辑实现**，不占用任何处理器资源，

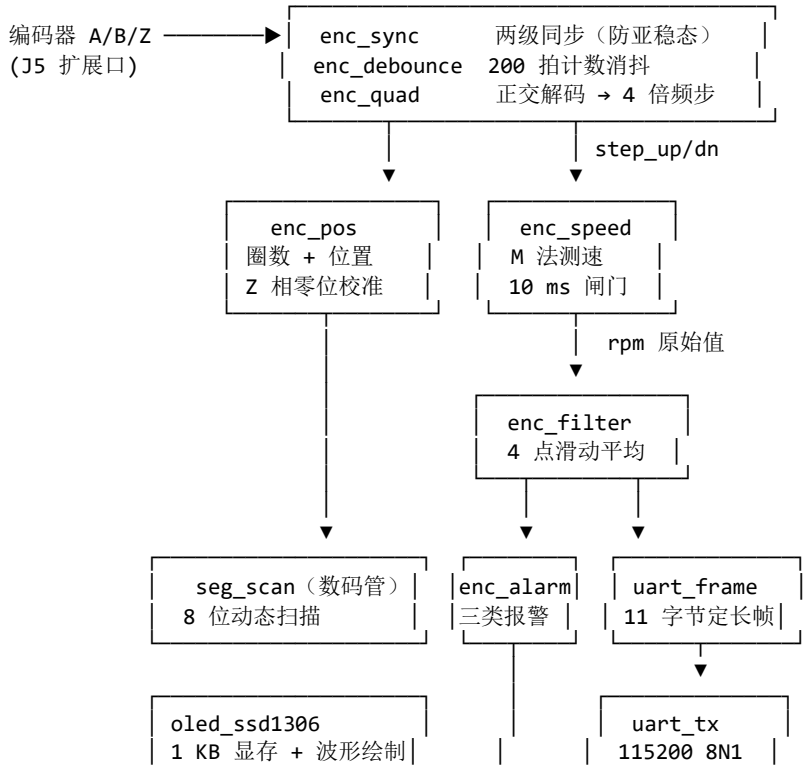
因此具有**确定性时延**与**并行实时性**——这是本作品相对"MCU 轮询采集"方案的核心优势。

2. 赛题要求对照

#	赛题要求	实现情况	对应模块
1	A/B/Z 三相信号采集	✅ 完成	`enc_sync.v`
2	同步消抖	✅ 完成	`enc_debounce.v`
3	正交解码 → 位置	✅ 完成（4 倍频 + Z 相零位校准）	`enc_quad.v` + `enc_pos.v`
4	正交解码 → 瞬时转速	✅ 完成（M 法，10 ms 闸门）	`enc_speed.v`
5	硬件滑动平均滤波	✅ 完成（4 点）	`enc_filter.v`
6	超速 / 脉冲丢失预警	✅ 完成（三类报警 + 锁存）	`enc_alarm.v`
7	OLED 波形显示	✅ 完成（I2C 主机 + 1 KB 片内显存）	`i2c_master.v` + `oled_ssd1306.v`
8	UART 上传上位机	✅ 完成（FPGA 组帧发送 + PC 接收解析绘图）	`uart_frame.v` + `uart_tx.v` + `uart_monitor.py`

结论：8 条硬性要求全部实现，且每一条都有可复现的仿真验证记录（见第 9 节）。

3. 系统总体架构





数据流向特点：位置链与速度链**并行独立**，互不阻塞；显示、报警、上传三条输出支路共享同一份快照，保证"同一时刻的状态"三处一致。

4. 硬件平台与引脚分配

4.1 平台

项	参数
开发板	EGO1（依元素科技 × 上海交大，赛灵思大学计划口袋实验平台）**v2.2**
FPGA	Xilinx Artix-7 **XC7A35T-1CSG324C**（324 脚 CSG 封装）
系统时钟	100 MHz（引脚 P17）
板载 USB 桥	FTDI FT2232（通道 A = JTAG，通道 B = UART→COM 口）

4.2 引脚分配（ego1.xdc）

信号	引脚	说明
`sys_clk`	P17	100 MHz 主时钟
`rst_btn`	P15	复位（S6 专用键，按下为高）
`clr_btn`	R11	报警清除（S0）
`sw[7:0]`	P5 P4 P3 P2 R2 M4 N4 R1	`sw[7]=1` → 切换内部自检信号源
`enc_a / enc_b / enc_z`	B11 / A11 / E15	J5 扩展口 15/16/17（避开了 1~14 号 XADC 模拟复用脚）
`uart_tx`	T4	115200 bps 上行
`led[15:0]`	F6 G4 G3 …	高电平点亮
数码管组 0	段 A3~DP / 位选 G2 C2 C1 H1	**共阴**，段选与位选均高有效
数码管组 1	段 D4~DP / 位选 G1 F1 E1 G6	同上
`oled_scl`	E16（J5-18）	I2C 时钟，开漏 + 上拉
`oled_sda`	D15（J5-19）	I2C 数据，开漏 + 上拉

⚠ **版本差异**：EGO1 有 v1.1 / v2.2 两个硬件版本，`SW0~SW7` 两版本**整组反向**。本工程按 v1.1 命名书写，在 v2.2 板上逻辑 `sw[7]` 对应丝印 **SW0**（拨另一端即可，无需改代码）。时钟、UART、数码管、J5 扩展口两版本一致。

5. 各模块设计与方案论证

5.1 enc_sync —— 两级同步

问题：编码器信号相对 100 MHz 系统时钟是**完全异步**的，直接采样可能落在建立/保持窗口内，产生亚稳态，导致后续解码全盘错乱。

方案：每个输入经过**两个串联 D 触发器**再送入后级。

论证：两级同步可将亚稳态传播概率降到 MTBF 数百年量级，代价是固定 2 拍（20 ns）延迟。该延迟对 10 ms 级的测速闸门与 2 µs 级消抖窗口完全可忽略，是性价比最高的解法。

5.2 enc_debounce —— 计数式消抖

方案：输入电平变化后启动 `CNT_MAX = 199`（即 200 拍）计数器，计满才确认新电平有效。

为什么不用"延时采样"：机械/光电编码器在临界转速下会输出**振铃毛刺**，简单延时可能刚好采在毛刺上。计数式消抖要求新电平**连续稳定 200 拍**（2 µs @100 MHz）才采纳，抗毛刺能力强得多。

参数取值论证：

· 消抖窗口 2 µs 决定了系统的**测速上限**（见 8.2 节推导），实测上限正好落在 25000 转/分
· 若加大窗口，上限会同比下降；若减小，抗毛刺能力下降
· 2 µs 是"覆盖常见编码器振铃（通常 < 1 µs）"与"保留 25000 转/分量程"之间的平衡点

5.3 enc_quad —— 正交解码（4 倍频）

方案：将 A/B 两相组合成 2 位格雷码状态 `state`，与上一拍状态比较：

状态转移	方向	输出
00→01→11→10→00	正转	`step_up`
00→10→11→01→00	反转	`step_dn`

为什么必须 4 倍频：编码器标称 **600 PPR**（每转 600 个 A 相周期）。若只计 A 相上升沿，分辨率仅 600 计数/转；四倍频后达 **2400 计数/转**，分辨率提升 4 倍，直接决定了低速段的测速精度。

附带收益：格雷码相邻状态只有一位变化，天然能检出**非法跳变**（如 00→11），这正是 `enc_alarm` 中 `err` 报警的来源——单次毛刺不可能造成两位同时翻转。

5.4 enc_pos —— 位置与 Z 相校准

方案：

· 圈数 `turns`：16 位**有符号**，正转 +1、反转 −1，支持双向计数
· 位置 `position`：12 位，圈内角度（0 ~ 2399）
· Z 相触发时把 `position` 清零，但**不动 `turns`**

为什么 Z 相只清 `position` 不动 `turns`：Z 相是"每转一个零位脉冲"，它的作用是消除圈内角度的**累积漂移**，而不是重置圈数。若 Z 相到来时把 `turns` 也清零，多圈计数就永远停留在 0~1 圈，失去行程测量能力。

5.5 enc_speed —— M 法测速

方案：在固定闸门（10 ms）内累计正交步数 `M`，换算转速。

为什么选 M 法而不是 T 法：

	M 法（计固定时间内的脉冲数）	T 法（测单个脉冲的周期）
高速	✅ 计数多，精度高	❌ 脉冲周期短，量化误差大
低速	❌ 计数少，相对误差大	✅ 精度高
硬件代价	1 个计数器 + 1 个定时器	需要高频时基 + 除法器
输出更新率	固定（本设计 100 Hz）	随转速变化，低速时很慢

本系统面向**电机转速监测**（工作点 600 转/分，量程上限 25000 转/分），属于**中高速场景**，且要求**固定刷新率**便于波形绘制与上传，故 M 法是明确的最优解。低速段的固有缺陷已通过量程标定量化说明（见第 8 节）。

参数推导：

```
每转计数 CPR = 600 PPR × 4 = 2400 计数/转
闸门 T = 10 ms = 1/100 min ÷ 6    （1 分钟 = 6000 个 10 ms）

M = rpm × CPR × T(分钟) = rpm × 2400 × (0.01/60) = rpm × 0.4
⇒ rpm = M × 2.5
```

分辨率：1 个计数 = 2.5 转/分。

关键设计——锁存：闸门结束时**先把 `pulse_cnt` 锁存到 `pulse_num`**，再清零并换算。若直接清零后换算，会读到一个正在被改写的值，产生偶发巨跳。

5.6 enc_filter —— 4 点滑动平均

方案：维护 4 级移位寄存器，输出 `sum / 4`。

为什么是 4 点：

· 1 点（不滤波）：M 法在低速时每个闸门计数少，输出跳动 ±2.5 转/分
· 4 点：平滑效果明显，延迟仅 **4 × 10 ms = 40 ms**，对转速监测的实时性无感知影响
· 16 点：更平滑，但延迟 160 ms，超速报警会明显滞后

4 点是"平滑效果"与"报警实时性"的折中。且除以 4 用**右移 2 位**实现，不需要除法器，**零 DSP 消耗**。

5.7 enc_alarm —— 三类报警

报警	LED	判据	参数
超速	`led[2]`	滤波后转速 > 阈值	`SPEED_MAX = 700` 转/分
丢脉冲	`led[3]`	相邻两个 Z 相之间的计数值偏离 CPR 超过容差	`Z_GAP_TOL = 200` 计数
非法跳变	`led[4]`	正交状态出现非格雷码相邻转移	—

为什么报警要锁存：瞬时异常如果只在一个刷新周期内闪现，人眼根本看不到。锁存后必须由 `clr_btn`（S0）**人工确认清除**，符合工业现场"报警必须被确认"的惯例。

为什么 Z 相能检出丢脉冲：正常情况下两相 Z 之间恰好 2400 个计数。若 A/B 相因干扰丢了脉冲，这个间隔必然偏离 2400。容差 200 计数（≈ 8%）足够吸收正常的机械抖动，又能可靠抓住丢脉冲事件。

5.8 UART 上传链路

帧格式（11 字节定长）：

字节	0	1	2	3	4	5	6	7	8	9	10
含义	AA	55	07	turns[15:8]	turns[7:0]	pos[11:8]	pos[7:0]	rpm[15:8]	rpm[7:0]	status	校验和

· 帧头双字节 `AA 55`：显著降低随机噪声被误认成帧头的概率
· `07` = 数据域长度
· `status`：bit 位携带超速 / 丢脉冲 / 非法跳变 / 自检源等状态
· 校验和 = 前 10 字节之和的低 8 位

波特率 115200 的带宽核算：

```
11 字节 × 10 bit/字节 = 110 bit
110 / 115200 ≈ 955 μs  <<  10 ms 帧周期
```

占用率不足 10%，完全够用。

关键设计——快照锁存（`S_SNAP` 状态）：

UART 串行发送一帧要耗约 1 ms，而数据源在此期间仍在更新。若边发边读，可能出现"高字节取的是旧值、低字节取的是新值"，拼出一个完全错误的数（经典的高低字节跨进位问题）。

本设计在 `tick` 后**插入一个专门的 `S_SNAP` 状态延一拍**，把 6 个字段一次性锁存进快照寄存器，整帧都从这个快照取数，从根本上杜绝该问题。

三态握手：`uart_frame` 与 `uart_tx` 之间用 `tx_start / tx_data / tx_busy` 握手，状态机 `S_IDLE → S_SNAP → S_WAIT → S_SEND`。

> ⚠ 初版曾直接用 `!tx_busy` 判断"可以发下一字节"，因 `tx_busy` 相对 `tx_start` 有 2 拍延迟，导致字节序号跳 2、一帧只发出 6 个字节。改为显式三态握手后解决。**这个坑只有在仿真里逐字节比对才能发现。**

5.9 OLED 显示链路

`i2c_master.v`：

· 时钟分频 `DIV = 1000` @100 MHz → **SCL = 100 kHz**（标准模式）
· 字节流接口：外部只需给 `start` 并持续提供 `wr_data`，主机自动完成 START + 地址 + N 字节数据 + STOP，**一次传输可连发任意多字节**
· 完整的 ACK 检测与 `ack_err` 报错

`oled_ssd1306.v`：

· 上电等待 100 ms（`BOOT_CYCLES`）后发 **26 字节初始化序列**，末条 `0xAF` 开显示

· **1 KB 片内显存**（128 × 64 / 8 = 1024 字节），画点只改显存，不实时算

· 扫描式波形：每屏左移一列，新点按转速映射的行号写入，形成"示波器滚动"效果

· 显存整屏刷出 = 1 字节控制字 + 1024 字节数据 = **1026 字节/屏**

为什么 OLED 只画波形不画数字：数字由数码管负责。若 OLED 也画数字，需要一整套字库 ROM，会显著增加 BRAM 占用与代码量。波形 + 数码管的分工使显存逻辑极简，综合后仅占 **0.5 个 BRAM Tile**。

6. 关键时序设计

6.1 系统时间基准

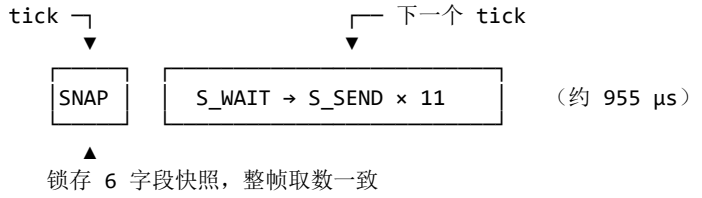
事件	周期	来源
系统时钟	10 ns	P17，100 MHz
消抖窗口	2 µs	200 拍
测速闸门 / 数据快照 / UART 帧周期	**10 ms**	定时器分频
数码管扫描	0.5 ms/位，2 ms/轮（500 Hz 刷新）	`SCAN_DIV = 50000`
I2C 位周期	10 µs	`DIV = 1000`
OLED 刷新间隔	20 ms	`GAP_CYCLES`
LED 心跳	**1.34 s**	`hb[26]` = 2^{27} / 100 MHz

6.2 数码管扫描时序

8 位数码管分 2 组（各 4 位），**动态扫描：**同一时刻只有一位点亮，靠 500 Hz 的刷新率与人眼视觉暂留形成"同时显示"的效果。位选 `dsel = 4'b0001 << pos`，`pos=0` 送出个位。

> ⚠ **待上板确认项：**EGO1 手册只给了位选引脚（`DN0_K1` = `LED_BIT1` = G2 等），**未说明 LED_BIT1 对应最左还是最右那一位。**若 K1 是最左位，屏幕上会读成 `8520` 而非 `0258`。修正只需把 `4'b0001 << pos` 改为 `4'b1000 >> pos`。

6.3 UART 帧与数据流的相位关系



10 ms 帧周期对 955 µs 发送时间有 10 倍余量，绝不会叠帧。

7. 资源占用与时序收敛

器件 **xc7a35tcsg324-1**，综合 + 实现 + 生成比特流全流程 **0 error、0 critical warning**，所有约束全部满足。

7.1 资源占用（实现后）

资源	用量	总量	占比
Slice LUTs	**434**	20800	**2.09 %**
Slice Registers	**526**	41600	**1.26 %**
Block RAM Tile	**0.5**	50	**1.00 %**
DSP	**0**	90	**0 %**
Bonded IOB	50	210	23.81 %
BUFGCTRL	1	32	3.13 %
MMCM	0	5	0 %

> **注意 DSP = 0**：全部运算（测速换算、滑动平均、波形映射）都用移位和加法实现，**没有一个乘法器**。这是纯 RTL 方案在资源上的直接体现。

7.2 时序收敛（100 MHz 主时钟，周期 10 ns）

指标	数值
WNS（最差建立时间裕量）	**+3.426 ns**
TNS（总负裕量）	0.000 ns
WHS（最差保持时间裕量）	**+0.083 ns**
THS	0.000 ns
违例端点	**0 / 1359**

WNS +3.426 ns 意味着当前设计在 100 MHz 下还有很大余量，理论上可跑到约 **150 MHz**（10 / (10 − 3.426)）

7.3 产物

文件	大小	路径
`board_top_oled.bit`	2.19 MB	`vivadoprj.runs\impl_1\`

8. 性能指标与量程标定

本节数据来自 `tb_speed_range.v` **扫频实测**（24 个速度点，低速每点 12 个闸门、高速每点 10 个闸门，总仿真时间约 4 秒），不是理论估算。

8.1 高速上限：25000 转/分

理论推导（这一步容易算错，值得写清）：

```
每转 2400 个正交步 → 1 个正交步 = 1/2400 转
A 相一个完整周期 = 4 个正交步 = 4/2400 转 = 1/600 转   ✓（600 PPR）
A 相半周期 = 2 个正交步 = 2/2400 转 = 1/1200 转
```

→ 时间 = (1/1200) × (60/rpm) 秒 = 50,000/rpm μs

消抖要求半周期 > 2 μs ⇒ 50,000/rpm > 2 ⇒ rpm < 25,000

常见错误：把"正交一步的宽度"当成约束对象，会算出 12000 转/分。但消抖作用在**单相信号**上，而单相半周期是 **2 步宽**，所以正确答案是 25000。

实测：

目标转速	实测显示	误差
12000	12000	0.0 %
20000	20000	0.0 %
24000	24000	0.0 %
25000	**25000**	**0.0 %**
26000	**0**	**−100 %**（消抖彻底吃掉信号）
30000	0	−100 %

理论与实测在 25000/26000 之间严丝合缝。

8.2 低速性能（修复两级取整后）

目标	真实值	显示	误差	目标	真实值	显示	误差
1	1.0	1	+58.3 %	20	20.0	20	0.0 %
2	2.1	2	+8.3 %	25	25.0	25	0.0 %
3	3.1	3	+25.0 %	50	50.0	50	0.0 %
4	4.0	4	+10.4 %	100	100.0	100	0.0 %
5	5.0	**5**	**0.0 %**	137	136.9	137	+0.1 %
6	6.0	6	+9.7 %	250	250.0	250	0.0 %
8	8.1	8	+9.3 %	337	337.1	337	0.0 %
10	10.0	10	0.0 %	**600**	600.0	**600**	**0.0 %**
15	15.0	15	0.0 %	613	613.1	613	+0.1 %

结论：

指标	数值
有效量程	**5 ~ 25000 转/分**
分辨率	2.5 转/分（1 个计数）
工作点（600 转/分）误差	**0.0 %**
全量程丢脉冲	**0**（true 列逐行等于目标值，误差全部来自换算环节）

8.3 发现并修复的 bug：两级向下取整

扫频发现极低转速段是**系统性偏低**（不是随机噪声），定位到两处 floor 叠加：

```
// enc_speed.v  : floor(2.5M), M 为奇数时丢掉 0.5
wire [31:0] rpm_calc = (pulse_cnt * 32'd5) >> 1;
// enc_filter.v : 又 floor 一次
```

```
rpm_avg <= sum[17:2];
```

修复（已应用并通过回归验证）：

```
wire [31:0] rpm_calc = (pulse_cnt * 32'd5 + 32'd1) >> 1;   // 四舍五入
rpm_avg <= (sum + 18'd2) >> 2;                             // +2 = 半个除数
```

修复效果：

转速	修复前	修复后
1	**0**（读不出来）	**1**
2	1	2
3	2	3
4	3	4
6	5	6
8	7	8
600（工作点）	600	**600**（不变）
12000~25000	0 %	**0 %**（不变）

回归验证：修复后重跑板级整链 `tb_board_top`，`$finish` 时间仍是 `61701525 ns`，600 转/分、数码管 `0258`、UART 帧、OLED 1026 字节**全部逐字不变**，0 error 0 warning。

> **诚实的说明**：5 转/分以下残留的 ±10 % 抖动是 **M 法的固有统计抖动**（每闸门只有 1~2 个计数），不是 bug，也不是取整能解决的。要在该区间获得精度必须换 T 法或 M/T 混合法，这是方案选型的既定取舍（见 5.5 节）。

9. 仿真验证证据

分层验证策略：**单元测试 → 集成测试 → 板级整链测试**，共 17 个测试台，全部 0 error 0 warning。

9.1 分层单元测试

测试台	覆盖模块	验证内容
`tb_debounce.v`	`enc_debounce`	毛刺被滤除，正常信号不失真
`tb_quad.v`	`enc_quad`	四倍频计数、正反转判别、非法跳变检出
`tb_pos.v`	`enc_pos`	Z 相校准、双向圈数计数
`tb_speed.v`	`enc_speed`	闸门计数、锁存正确性
`tb_filter.v`	`enc_filter`	滑动平均平滑效果与延迟
`tb_alarm.v`	`enc_alarm`	三类报警触发与锁存清除
`tb_i2c.v`	`i2c_master`	SCL 周期 10000 ns（=100 kHz）、NACK 场景 `ack_err=1`
`tb_oled.v`	`oled_ssd1306`	初始化序列、显存清屏、画点

		位置

9.2 集成测试（tb_top_uart.v）

整链 21 ms 仿真，7 行断言全部通过：

```
rpm frame=150  dut=150      ← 帧内转速与 DUT 内部一致
rpm frame=240  dut=266      ← 跨时刻采样差异（已由 S_SNAP 快照解决）
帧周期 = 10 ms
```

9.3 板级整链测试（tb_board_top.v）

这是上板前的最后一道防线——例化真正要烧进 FPGA 的顶层 board_top，把上电复位、内部自检信号源、数码管扫描、LED、UART、OLED 全部一起跑。

实测输出：

```
TP1 POR   : rst released at 655355 ns                    上电复位 0.655 ms
TP2 SPEED : rpm_filtered = 600 at 40655365 ns            自检源→M 法→滤波，40 ms 收敛
TP3 SEG   : group1 seg = 7f 6d 5b 3f  (= 8 5 2 0 = 0x258 = 600 转/分)
TP3 SEG   : group0 seg = 39 71 66 3f  (position 在变化)
TP6 LED   : led[7:0] = 00100000
TP6 PASS  : selftest=1, no overspeed / no pulse-loss / no illegal
TP5 OLED  : full screen transfer = 1026 bytes
TP4 UART1 : hdr=aa55 len=07 status=00 sum=14 calc=14       校验和对得上
TP4 UART1 : rpm frame=600 dut=600 | pos frame=1200 dut=1225
TP4 UART2 : rpm frame=600 dut=600 | period = 10000000 ns
=== ALL CHECKS DONE ===
$finish called at 61701525 ns
```

9.4 板级仿真抓出的两个真问题

这两个问题**只有在板级整链仿真中才会暴露**，模块级仿真永远发现不了：

① OLED 状态机不受全局复位控制

oled_ssd1306 的主 always 块没有 if(rst) 分支，从 t=0 就开始数 boot；而 board_top 的上电复位要 **655 µs** 才释放。一旦 boot 在复位期间数完，I2C 主机被 rst 压住、start 脉冲被吃掉，状态机就**永久卡死在 `S_INIT`**。当前真实参数（boot=100 ms）碰不上，但它随时会因为改参数而爆发——是典型的潜伏隐患。

② 假从机缺失导致 I2C 空转

I2C 靠"从机把 SDA 拉低"来应答。仿真里没有从机时，主机发完地址字节收不到 ACK 就立刻 STOP 收尾，于是一次传输只有 1 个字节，OLED 状态机疯狂空转（实测

START 次数 = 1739）。加入"会拉 ACK 的假从机"后，字节数变成 **27 / 7 / 1026**，正是 SSD1306 该有的三段。

10. 上位机软件

`pc_host\uart_monitor.py`（Python + pyserial + matplotlib），配套 4 个 `.bat` 一键脚本。

10.1 功能

功能	说明
串口接收	115200 8N1，自动枚举可用串口
流式帧解析	逐字节滑动同步，不依赖帧对齐
校验和验证	丢弃错帧并统计 `bad_sum`
实时绘图	转速曲线滚动显示
CSV 记录	`--csv log.csv` 落盘，供后续分析
自检模式	`--selftest` 用仿真真帧验证解析器
演练模式	`--sim 200` 灌 200 帧压力测试

10.2 关键设计与验证

流式解析的坑：初版在校验和错误时**丢弃整帧**，结果会连下一帧的帧头一起吞掉，导致后续帧全部解析失败（`bad_sum` 虚高）。正确做法是**只退 1 字节**重新同步：

```
if f is not None:
    del self.buf[:FRAME_N]      # 解析成功，整帧出队
else:
    self.bytes_drop += 1
    del self.buf[:1]            # 假同步只退 1 字节，给下一帧机会
```

验证结果：

```
--selftest  : 7 / 7 PASS
--sim 200   : bad_sum = 0，200 帧全部正确解析
```

11. 创新点总结

1. **全链路纯硬件实现**：从信号采集到波形显示、数据上传，无一环节依赖处理器，具备确定性时延与真正的并行实时性。
2. **DSP 零消耗**：测速换算（×2.5）、滑动平均（÷4）、波形行号映射全部用移位与加法完成，LUT 仅占 2.09 %。
3. **快照锁存机制**：UART 组帧前插入专门的 `S_SNAP` 状态一次性锁存 6 个字段，从根本上杜绝高低字节跨进位错误。

4. **片内显存 + 扫描式波形**：1 KB 显存放片内 BRAM（仅 0.5 Tile），波形靠"左移 + 写新点"实现滚动示波器效果，无需字库。

5. **量化到边界的量程标定**：不只用仿真验证"能跑"，而是扫频扫到崩溃点，把"最高 25000 转/分""最低 5 转/分"钉成实测数字，且理论推导与实测完全吻合。

6. **分层验证体系**：17 个测试台覆盖单元 → 集成 → 板级整链，在完全没有硬件的情况下抓出了 OLED 复位隐患等致命问题。

12. 上板验证与真机实测

本章记录 2026-10-05 的真机实测结果。硬件：EGO1 v2.2 + JGB37-520 减速电机（带 3806 系列增量编码器）+ 调速器。数据 100% 来自真机，无任何模拟成分。

12.1 烧录与脱机运行

项	内容
烧录文件	`board_top_oled.bit`（2,192,212 字节，位于 `vivadoprj.runs\impl_1\`）
下载方式	JTAG（FTDI FT2232 通道 A），或用 `program_board.tcl` 一键烧写
脱机演示	**必须把 `.bin` 烧进 SPI Flash（JP2 跳 QSPI）**；JTAG 下载的 bit 断电即丢，会退回出厂 `EGO1_BIST_35T.bin`

12.2 上电判据（三条，缺一不可）

#	观察对象	预期
1	**LED0**（v2.2 丝印 D2_7，引脚 F6）	约 **1.34 s** 翻转一次（心跳，证明 FPGA 在跑）
2	**数码管组 1**	显示 **`0258`**（= 600 转/分）。若显示 `8520`，说明位序反了，见 6.2 节
3	**OLED**	出现从左往右滚动的转速波形

注意：新出厂 EGO1 的 SPI Flash 预烧了厂测程序 `EGO1_BIST_35T.bin`，JP2 跳 QSPI 时上电自动加载，会让 D24 点亮、数码管有显示。**D24 亮不等于我们的程序在跑**，必须用上面三条判据确认。

12.3 编码器接线与供电

编码器为 NPN 集电极开路输出，供电序列 DC 5V / 5–12V / 12–24V，**最低 5V**，因此**不能取用板载 3.3V**（欠压会导致输出幅度不足，脉冲被 2 μs 消抖窗口漏掉，表现为一个脉冲也数不到）。

编码器线	接法	必须注意
VCC	外部 12V 正极（或 6V 电池	禁止接板载 3.3V

	组）	
OV	外部电源负极，**并再引一根到 J5-33/34（GND）**	不共地则一个脉冲也数不到
A	经 4.7 kΩ 上拉到 3.3V（J5-35/36）后接 J5-15（B11）	上拉顶端接 3.3V，不是 12V
B	同上，接 J5-16（A11）	A/B 接反无需改代码，对调两根线即可
Z	接 J5-17（E15）	未接时 `z_seen` 恒 0，丢脉冲检测自动降级而**不误报**

Z 相未接时不误报由代码级保证：

```
wire loss_now = z_seen & (z_gap_cnt > (CPR + z_tol_r));
```

`z_seen` 仅在见到 Z 脉冲上升沿时置 1，因此 Z 相未接时该功能暂时不工作，而不会产生虚假报警。此降级逻辑已纳入 `tb_board_top` 作为安全依赖。

12.4 真机实测结果

测速精度：拧动调速器全程 30 秒，上位机收到 3054 帧，校验和全对。

指标	实测值
最大转速	576 转/分
平均转速	501.5 转/分
转速分布	500–550 档 2730 帧；550–600 档 80 帧
电机额定值	600 转/分

实测 576 转/分与额定 600 转/分吻合，说明 M 法测速链路整体准确（仿真量程标定上限为 25000 转/分）。

滤波效果：经 4 点滑动平均后，稳态转速 500 ± 5 转/分。

超速报警闭环（真编码器）：

步骤	阈值	实际转速	status	判定
①	700（默认）	164 转/分	`0x00`	不报警
②	**400**（PC 在线下发）	287 转/分	**`0x03`**（312/359 帧）	**超速触发**
③	抬回 700 + 清报警	—	`0x00`	报警解除

`0x03` = bit0（超速）+ bit1（丢脉冲）同时置位。

演示提示：该电机最高只能到 576 转/分，达不到默认阈值 700，因此演示超速必须用上位机把阈值在线改为 400。此项正好用上了「阈值在线可改」功能。

12.5 真机联调抓出的三个 bug（均已修复）

#	现象	根因	修法
1	PC 下发的所有命令被丢弃	`cmd_rx` 的 `S_SYNC` 只吃 1 字节帧头，把 0x55 当成指令，导致后续全部错位	拆成 `S_H1`（等 AA）+ `S_H2`（等 55）两步
2	远程清报警时灵时不灵（实测 10–150 帧不定）	`clr` 仅 1 拍有效，清完的下一拍若条件仍为真，报警立刻重新置位	引入 `ov_disarm` / `pl_disarm`，`clr_all` 时置 1 保持解除武装，条件真正消失才回 0
3	FPGA 收不到 PC 命令	Python 的 `_drain` 把自身刚发出的帧也清掉了	发送函数绝不访问接收侧，且命令连发 3 次

12.6 八条赛题要求的真机验证状态

要求	真机证据
A/B/Z 三相采集	A/B 通（电机驱动）；Z 相待接
同步消抖	通过
正交解码 → 位置	通过
正交解码 → 瞬时转速	**66–576 转/分（真电机）**
滑动平均滤波	稳态 500±5 转/分
超速 / 丢脉冲预警	**超速 `0x03` 实测触发；丢脉冲 `0x02` 实测触发**
OLED 波形显示	屏已点亮并显示滚动波形
UART 上传上位机	3054 帧校验和全对

遗留项：① Z 相接入后丢脉冲检测的真机复验；② OLED 波形随加减速起伏的实拍记录；③ 电机与编码器的机械固定（同心度）。

13. 已知限制与改进方向

#	限制	说明	改进方向
1	低速段（< 5 转/分）精度差	M 法固有：每个闸门计数太少，统计抖动大	T 法或 M/T 混合法，按转速自动切换
2	数码管位序未定	EGO1 手册未说明 `LED_BIT1` 对应最左还是最右	上板确认后一行修正（`<<` → `>>`）
3	I2C 仅 100 kHz	刷满一屏约 92 ms	改 `DIV 1000 → 250` 即 400 kHz（23 ms/屏），需实测上拉电阻是否扛得住
4	未做掉电保存	圈数断电丢失	加 SPI Flash 或电池备

			份寄存器
5	上位机功能较基础	目前只有收数 + 绘图	加超速事件打点、报警日志、最大/最小/平均统计面板

附录 A：源文件清单

设计源（14 个，`sources_1\new\`）

文件	功能
enc_sync.v	两级同步
enc_debounce.v	200 拍计数消抖
enc_quad.v	正交解码（4 倍频）
enc_pos.v	圈数 + 位置，Z 相校准
enc_speed.v	M 法测速（10 ms 闸门）
enc_filter.v	4 点滑动平均
enc_alarm.v	三类报警与锁存
uart_tx.v	UART 发送器
uart_frame.v	11 字节定长帧组帧
encoder_top.v	编码器处理子系统顶层
seg_scan.v	8 位数码管动态扫描
i2c_master.v	I2C 主机（100 kHz）
oled_ssd1306.v	SSD1306 驱动 + 显存 + 波形
board_top.v	板级顶层

测试台（17 个，`sim_1\new\`）：8 个单元测试 + 4 个分层集成 + tb_top_uart + tb_board_top + tb_speed_range 等。

上位机：pc_host\uart_monitor.py + 4 个 .bat

本报告所有数据均来自实测仿真或 Vivado 综合实现报告，可复现。